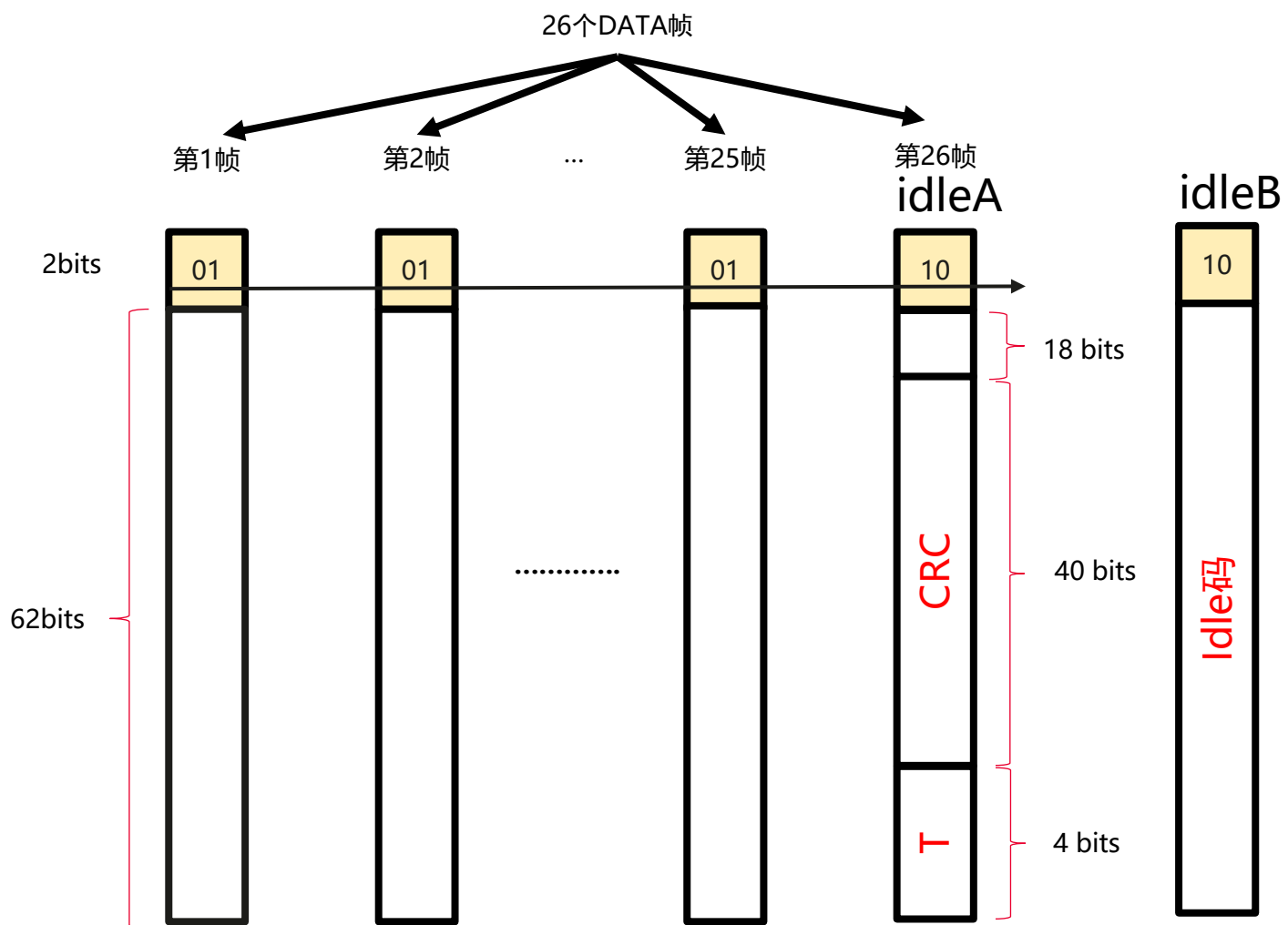


数字协议





加扰前

Idle码: $2C + BC * 7$ [61:0]

T: 0011 [3:0]

Data[63:62] = 01

idle[63:62] = 10

Total data : $196 * 8 = 1568 \text{ bits}$

62/64: $1568 / 62 = 26 \text{ frame}$

Redundant bits : $62 * 26 - 1568 = 44 \text{ bits}$

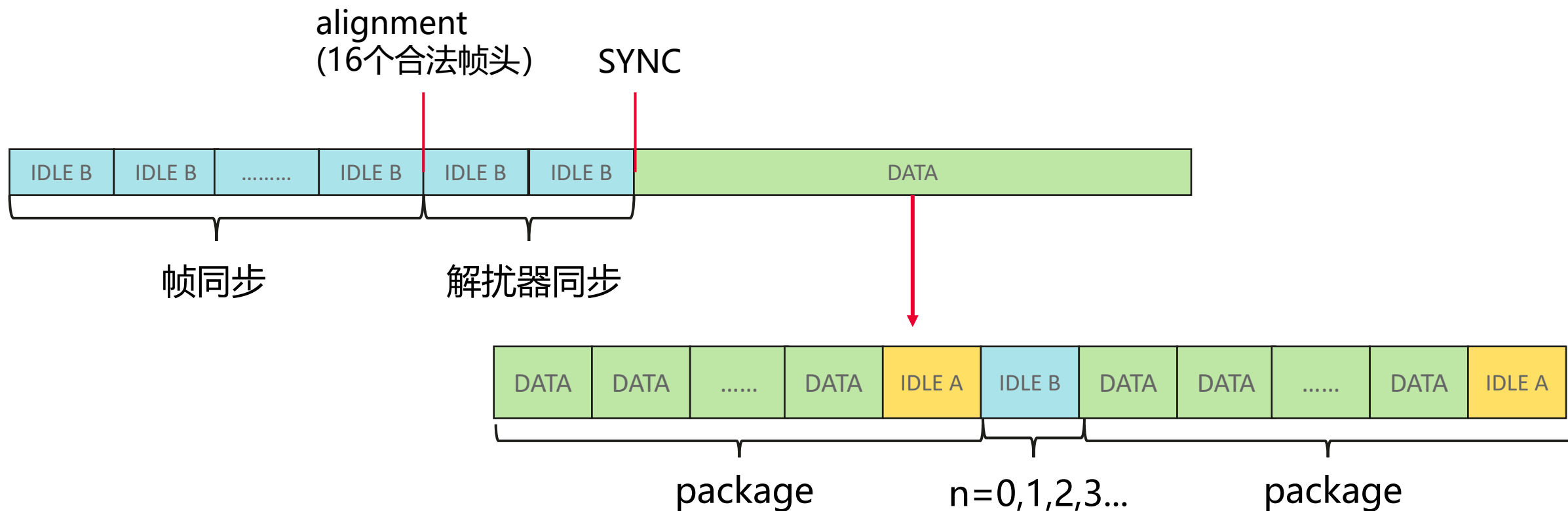
Data overhead: $1 - [1568 / (64 * 26)] = 6\%$

cap array max rate: 7.5Mhz

$$2*M+16*N+2$$

2: 连续2帧出错即失步 (每帧检测), 可配置为 $2*M$, M 为2bit
16: 连续16个合法帧头同步(01和10), 可配置为 $16*N$, N 为8bit
2: 解扰同步 (没有实际控制模块, 理论上最多两帧同步)

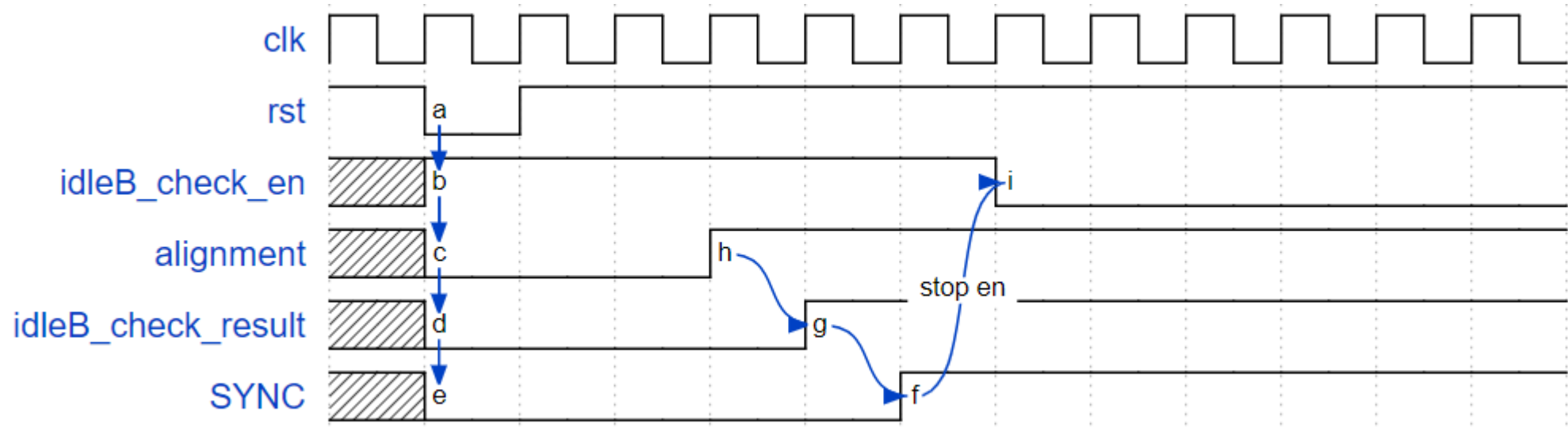
M, N 由SPI寄存器进行配置, 5个通道
共用同步/失步参数



Tips:

- ①失步后TX端不会发送IDLE B来进行同步。TX端只会在上电复位阶段发送大量IDLE B进行同步。
- ②数据包的值在IDLE A帧尾刷新。
- ③IDLE B发送期间, 输出数据保持前一个包的数据。

解扰正确判定



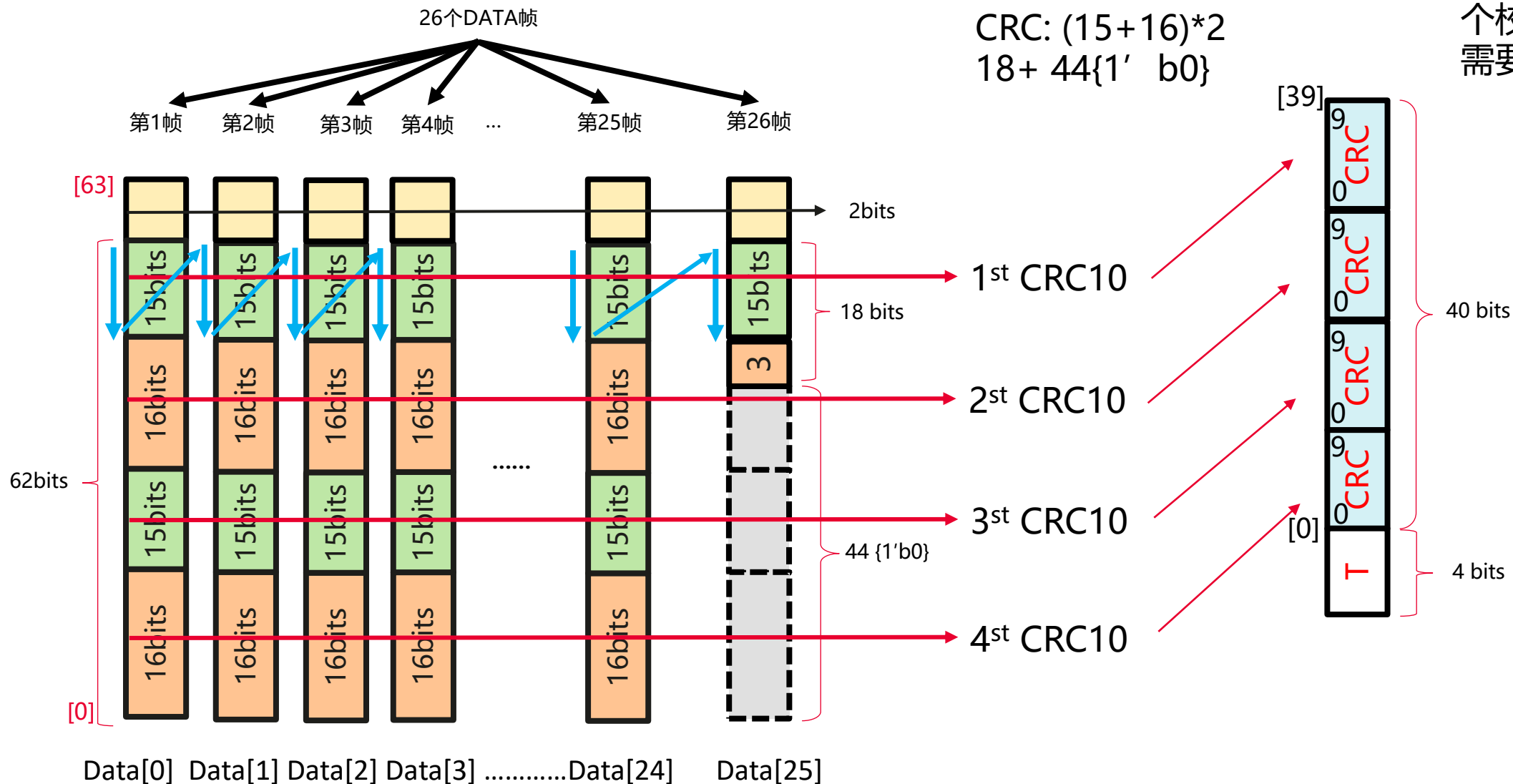
alignment: 数据流对齐

idleB_check_result: 数据流对齐后第一个IDLEB的解扰结果正确性(SYNC后保持常1, FSM)

SYNC同步: $SYNC = alignment \ \&\& \ idleB_check_result$

idleB_check_en: 复位时使能启动, 第一次SYNC同步时使能关闭

Tips: CRC只需要一个校验告警信号, 不需要四个分开告警

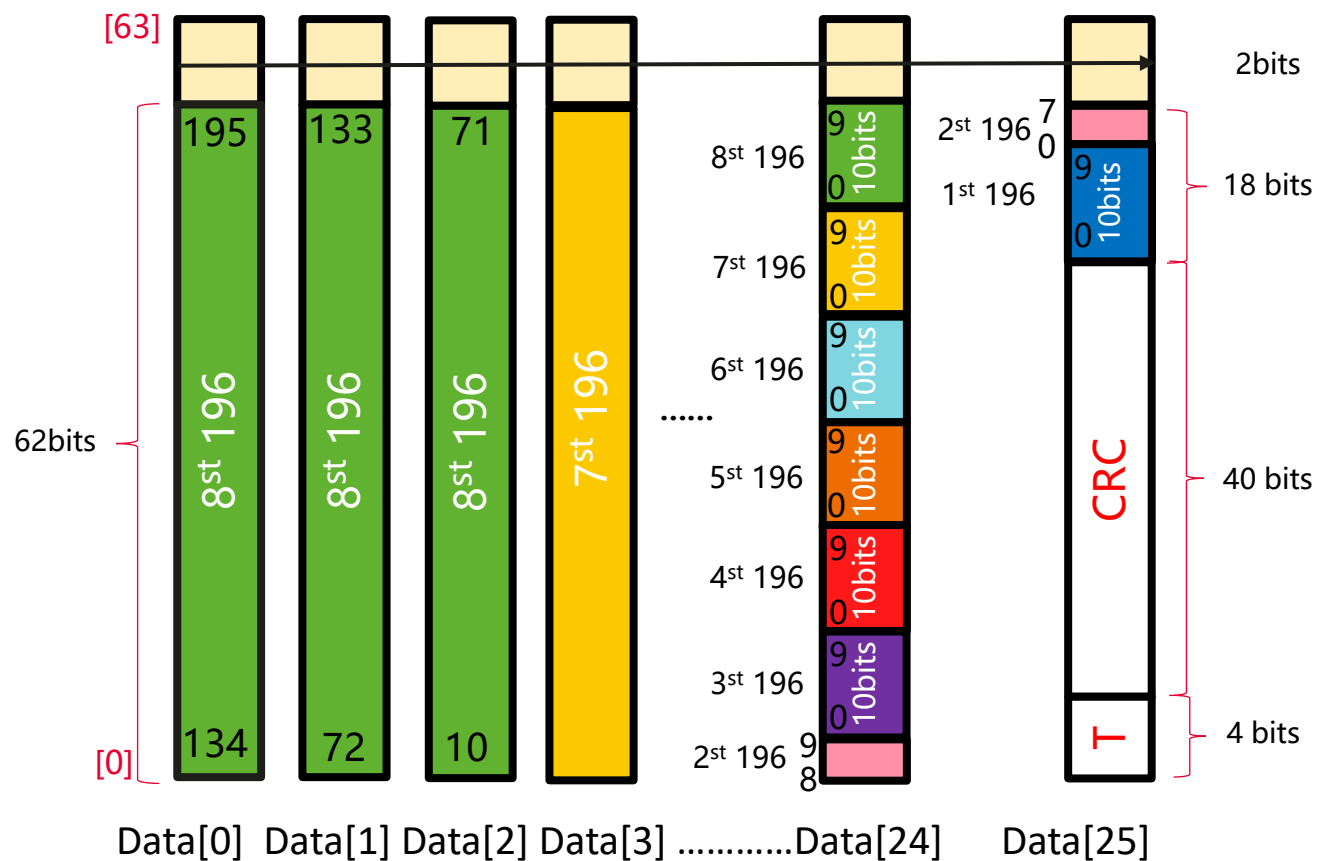
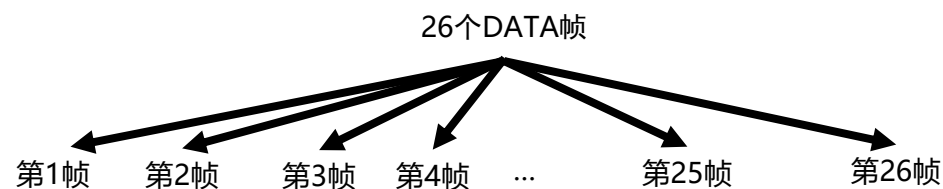


$$\text{CRC: } (15+16)*2 \\ 18 + 44\{1'b0\}$$

CRC10

$$0x319 = x^{10} + x^9 + x^5 + x^4 + x + 1 \text{ (CCITT)}$$

Tips: crc的初始值为全1, 即3FF。



4CRC10

- 1st 196 → {DATA[0], DATA[1], DATA[2], DATA[24][61:52]}
- 2st 196 → {DATA[3], DATA[4], DATA[5], DATA[24][51:42]}
- 3st 196 → {DATA[6], DATA[7], DATA[8], DATA[24][41:32]}
- 4st 196 → {DATA[9], DATA[10], DATA[11], DATA[24][31:22]}
- 5st 196 → {DATA[12], DATA[13], DATA[14], DATA[24][21:12]}
- 6st 196 → {DATA[15], DATA[16], DATA[17], DATA[24][11: 2]}
- 7st 196 → {DATA[18], DATA[19], DATA[20], DATA[24][1:0], DATA[25][61:54]}
- 8st 196 → {DATA[21], DATA[22], DATA[23], DATA[25][53:44]}

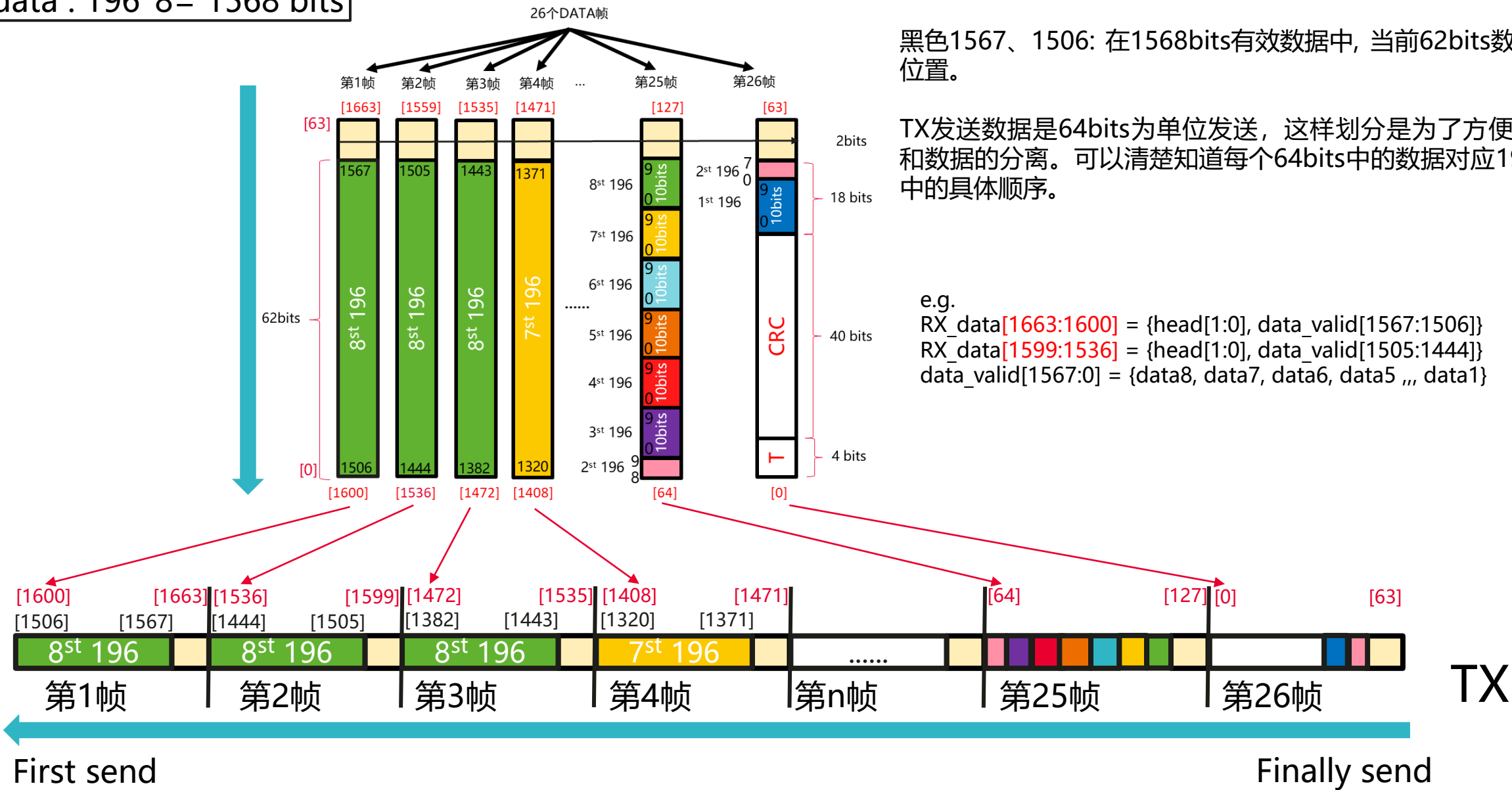
Total bits : 64*26=1664 bits
Total data : 196*8= 1568 bits

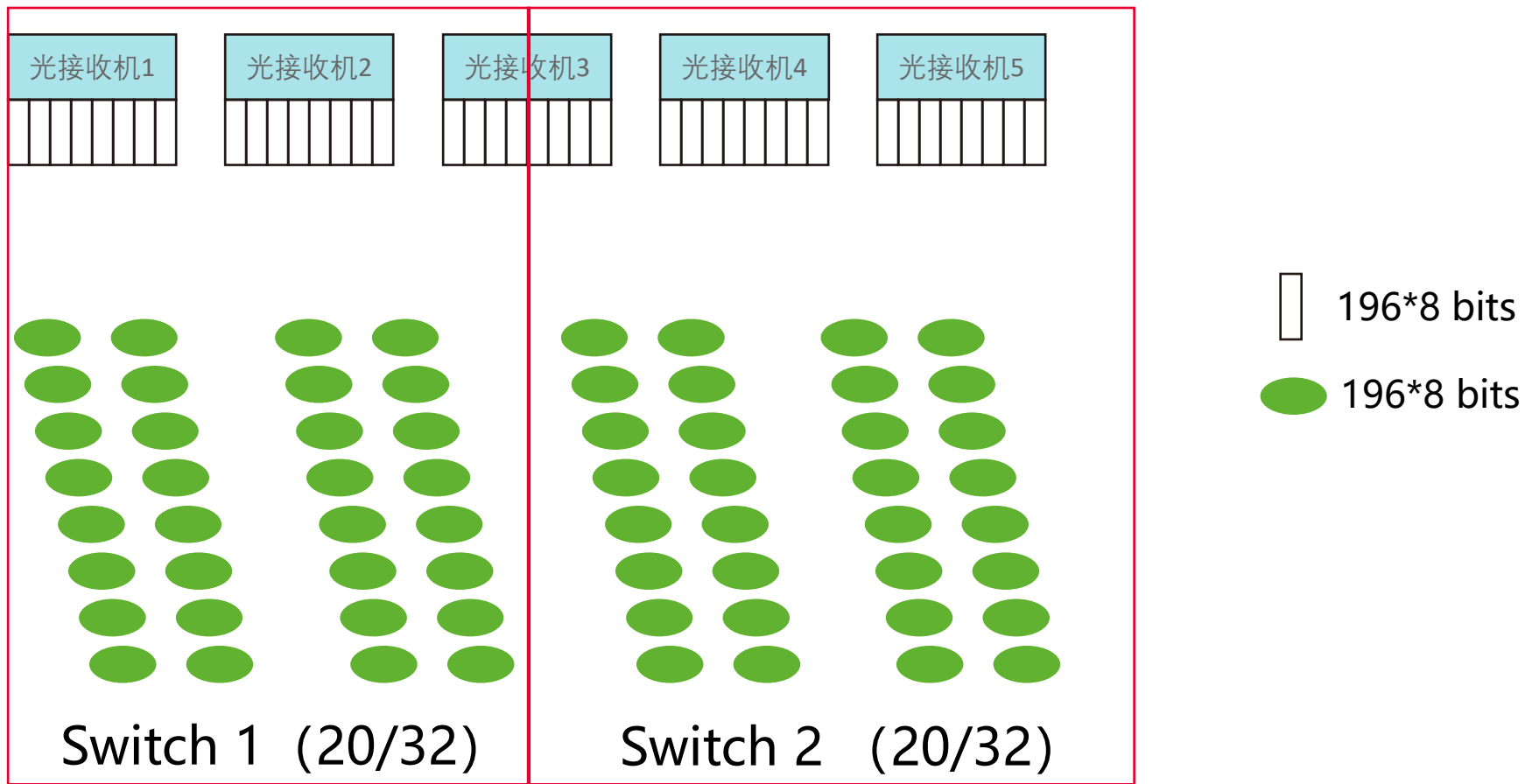
红色[1663][1600]: 在1664bits中, 当前包尾/包头的位置。

黑色1567、1506: 在1568bits有效数据中, 当前62bits数据的位置。

TX发送数据是64bits为单位发送, 这样划分是为了方便帧头和数据的分离。可以清楚知道每个64bits中的数据对应196*8中的具体顺序。

e.g.
RX_data[1663:1600] = {head[1:0], data_valid[1567:1506]}
RX_data[1599:1536] = {head[1:0], data_valid[1505:1444]}
data_valid[1567:0] = {data8, data7, data6, data5 ..., data1}





Switch输出5种模式(以196bits为单位)

1.正常模式 (没被选通的196bits置1)

2.196bits全1

3.196bits全0

4.196bits中第98bit置1, 其余全0

5.196bits中第98bit置0, 其余全1

000: 正常

111: 全1

100: 全0

110: 98为1, 其余全0

101: 98为0, 其余全1

TX design

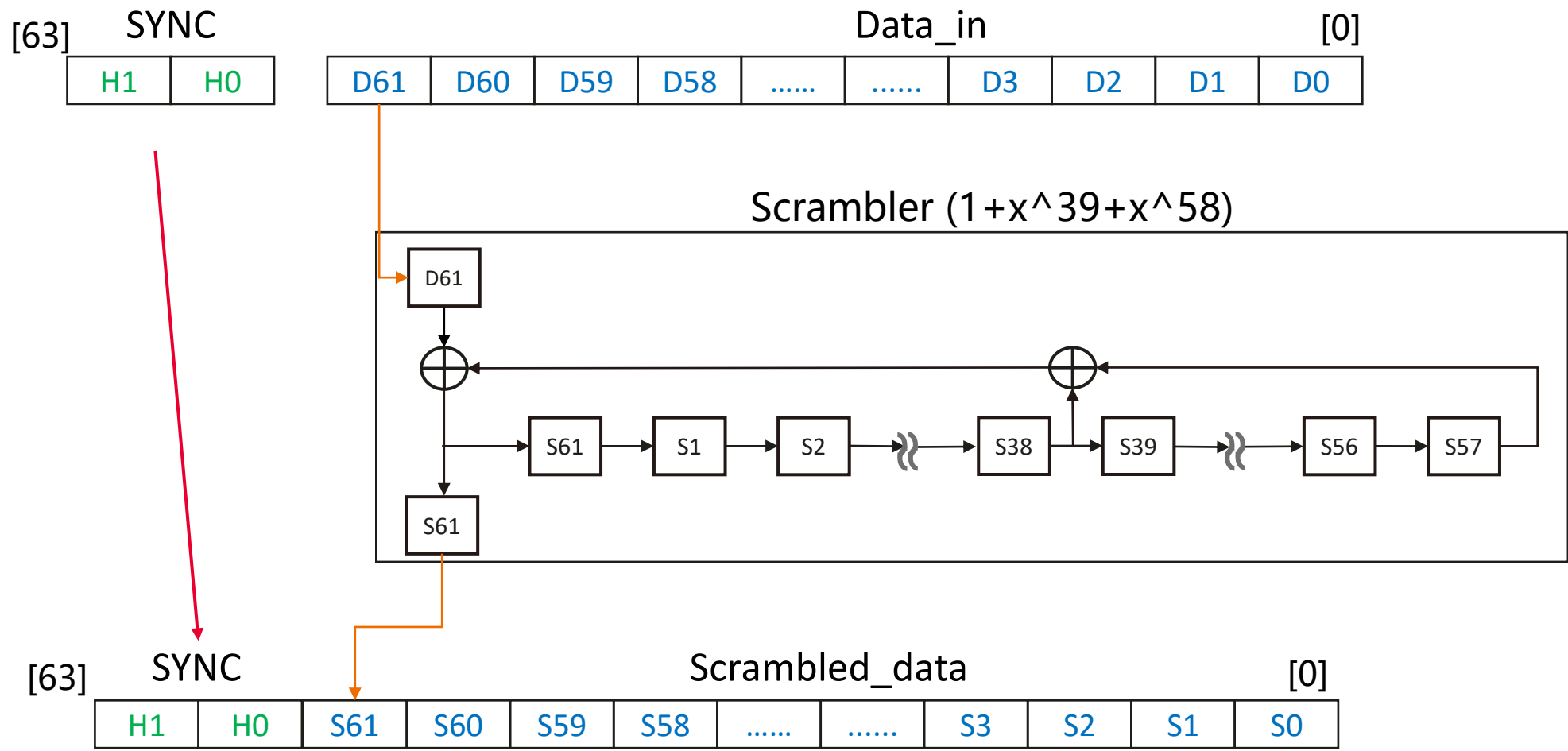
Initial seed:
58'h155_5555_5555_5555

方向：高位先加扰

Data:01
Idle:10

PRBS31,小端加扰

Vivado: aurora 64B66B IP
Descrambler(RX)/ Scrambler(TX)



RX design

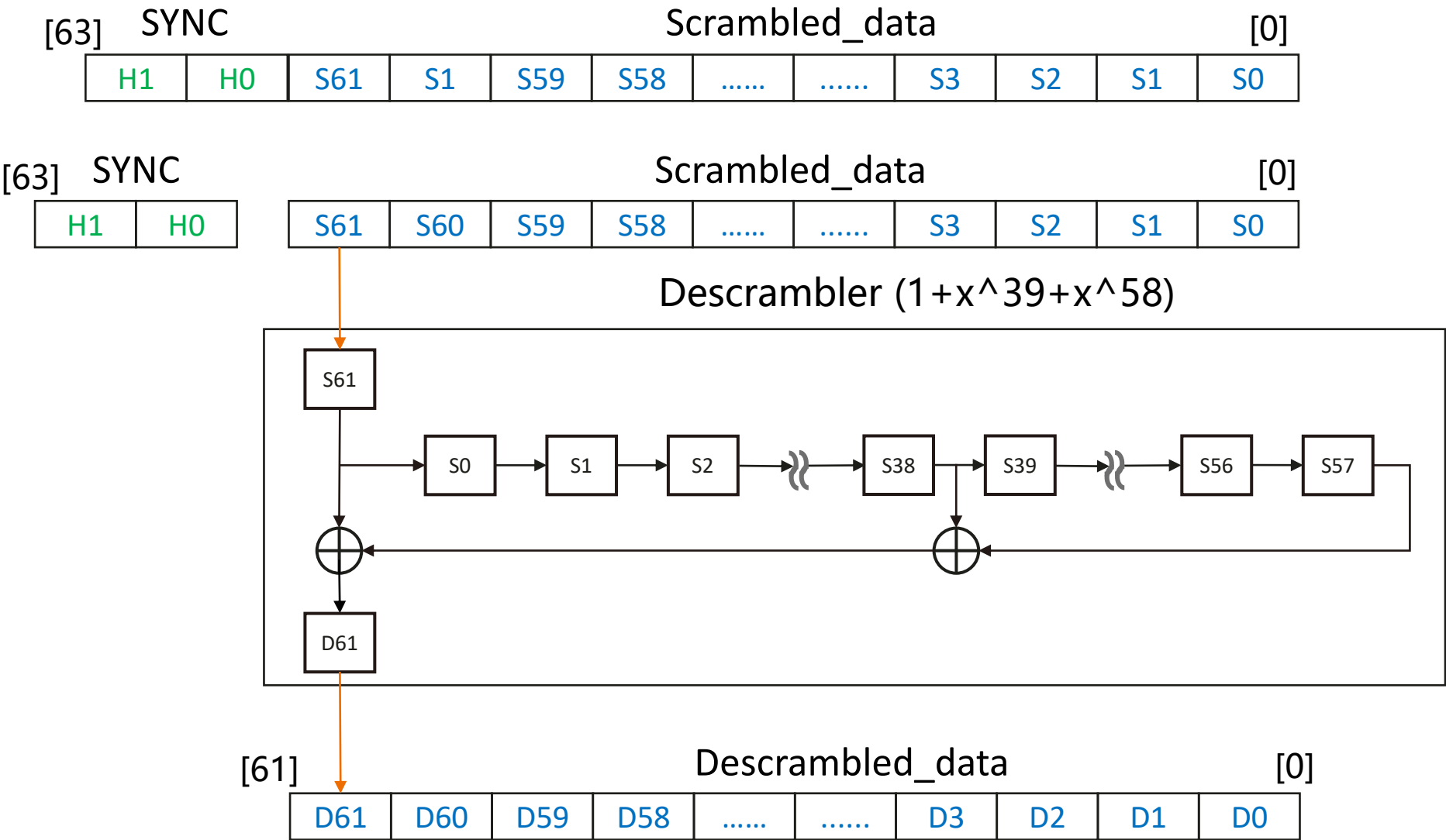
Initial seed:
58'h155_5555_5555_5555

方向：高位先解扰

Data:01
Idle:10

PRBS31,小端加扰
(IP信息见Page11)

Vivado: aurora 64B66B IP
Descrambler(RX)/ Scrambler(TX)



TX

[63:0] H1 H0 D61 D60.....D2 D1 D0

D0先发，存低位

RX Analog

[63:0] H1 H0 D61 D60.....D2 D1 D0

Basic Physical Resources Optional Features **Structural Options**

Simplify transceiver usage by organizing resources and helper blocks

Include transceiver COMMON in the

Include simple transmitter user clocking network in the

Include simple receiver user clocking network in the

Include reset controller in the

Include user data width sizing in the

Include transmitter buffer bypass controller in the

Include receiver elastic buffer bypass controller in the

Include In-System IBERT core

Expose additional ports by functionality, for advanced feature usage

All Ports

Select All Select None

Search: (6 matches)

Enable	Name	Information
✓	rxprbscntreset_in	RXPRBSCNTRESET on CHANNEL primitive
✓	rxprbsssel_in	RXPRBSSEL on CHANNEL primitive
✓	txprbsforceerr_in	TXPRBSFORCEERR on CHANNEL primitive
✓	txprbsssel_in	TXPRBSSEL on CHANNEL primitive
✓	rxprbserr_out	RXPRBSERR on CHANNEL primitive
✓	rxprbslocked_out	RXPRBSLOCKED on CHANNEL primitive

TX端口IP使用（Vivado）:

1.PRBS使用的是 Transceivers Wizard IP里面的（配置PRBS模式，样例工程可直接仿真）

2. aurora 64B66B IP Descrambler(RX)/ Scrambler(TX)

Thank you.